PIPELINE OPERATIONS GUIDE

AIOps PLE Platform

941K Users × 13 Tasks × 7 Shared Experts

대상: ML 엔지니어 (운영자)

2026-04-01 | Config-Driven Architecture

목차

1. 시스템 요구사항 ... 4
   1.1. 하드웨어 ... 4
   1.2. 소프트웨어 ... 4
   1.3. AWS 자격 증명 ... 4
   1.4. Docker 환경 ... 4
   1.5. 배포 환경 ... 5
   1.6. 디렉터리 구조 (핵심) ... 5
2. Phase 0: 피처 엔지니어링 ... 6
   2.1. 실행 흐름 ... 6
   2.2. Adapter 실행 ... 6
   2.3. PipelineRunner — 전처리 파이프라인 ... 6
      2.3.1. 3단계 정규화 (필수 이해) ... 6
   2.4. Generator 입력 라우팅 ... 7
   2.5. FeatureRouter — Expert별 피처 서브셋 라우팅 (활성화됨) ... 7
   2.6. Phase 0 출력 검증 (Pre-flight Check) ... 7
   2.7. SageMaker Processing Job (클라우드) ... 8
3. Phase 1–3: 학습 + Ablation ... 9
   3.1. 학습 아키텍처 개요 ... 9
   3.2. 모드별 실행 방법 ... 9
      3.2.1. 모드 1: 로컬 직접 실행 (개발/디버깅) ... 9
      3.2.2. 모드 2: Docker 로컬 실행 (SageMaker 환경 재현) ... 9
      3.2.3. 모드 3: SageMaker 클라우드 실행 ... 10
   3.3. Ablation 실행 (4-Dimension, 23 시나리오) ... 10
      3.3.1. Ablation 구조 ... 10
      3.3.2. 로컬 Ablation 실행 ... 11
      3.3.3. SageMaker Ablation 실행 (병렬) ... 11
      3.3.4. Ablation Hyperparameters ... 11
      3.3.5. Ablation 기본 학습 파라미터 ... 11
4. Phase 4: 지식증류 ... 13
   4.1. Teacher → LGBM Student ... 13
   4.2. 실행 CLI ... 13
   4.3. 증류 설정 (pipeline.yaml) ... 13
   4.4. Fidelity Gate ... 14
   4.5. LGBM 피처 중요도 기반 피처 선택 (증류용) ... 14
5. Phase 5: 서빙 ... 15
   5.1. 3단계 확장 아키텍처 ... 15
   5.2. Lambda 배포 ... 15
   5.3. 추론 흐름 ... 15
   5.4. 추천사유 생성 (Reason Generation) ... 15
   5.5. A/B 테스트 ... 16
6. Config 가이드 ... 17
   6.1. Split-Config 구조 ... 17

6.2. pipeline.yaml 주요 설정 ... 17
6.2.1. tasks 섹션 ... 17
6.2.2. training 섹션 ... 17
6.2.3. data 섹션 ... 18
6.2.4. aws 섹션 ... 18
6.2.5. cold_start 섹션 ... 18
6.3. feature_groups.yaml 핵심 구조 ... 19
6.4. task_groups 섹션 (GradSurgery / 태스크 유형 프로젝션 실험) ... 19
6.5. task_relationships (Logit Transfer) ... 20
7. 모니터링 ... 21
7.1. 데이터 드리프트 ... 21
7.2. 모델 성능 모니터링 ... 21
7.3. Fairness 감사 ... 21
7.4. 리니지 추적 ... 21
7.5. Champion/Challenger 평가 (오프라인 게이트) ... 22
8. 문제 해결 (FAQ) ... 23
8.1. Phase 2 학습 중 NaN Loss 발생 ... 23
8.2. GPU Utilization 낮음 (< 50%) ... 23
8.3. Label Leakage 의심 ... 23
8.4. SageMaker Job Timeout ... 24
8.5. Spot Instance 중단 빈번 ... 24
8.6. Gradient Norm 폭발 ... 24
9. 비용 관리 ... 26
9.1. Spot Instance 전략 ... 26
9.2. 비용 확인 CLI ... 26
9.3. Budget Guard ... 26
9.4. 비용 최적화 체크리스트 ... 26
9.5. 인프라 비용 비교 ... 27
9.6. 오케스트레이션 비용 ... 27
10. 부록: 운영 규칙 요약 ... 28
10.1. 절대 금지사항 ... 28
10.2. 관심사 분리 원칙 ... 28
10.3. 개발 순서 (필수 준수) ... 28
10.4. 체크포인트 재개 (Checkpoint Resume) ... 28
10.5. Pipeline State 자동 복구 ... 29
11. 운영/감사 에이전트 연계 ... 30

시스템 요구사항

하드웨어

구분	로컬 개발	AWS 클라우드
GPU	RTX 4070 12GB (또는 동급)	g4dn.xlarge (T4 16GB)
RAM	64GB 이상	ml.m5.2xlarge (32GB) — Phase 0
Storage	SSD 100GB+	S3 (무제한)
CPU	8코어 이상	자동 프로비저닝

소프트웨어

```
Python 3.10+
PyTorch 2.1+ (CUDA 12.1)
DuckDB 1.0+
cuDF (선택, GPU 가속 시)
Docker 24+ (로컬 ablation)
AWS CLI v2 + SageMaker SDK
```

AWS 자격 증명

```
# AWS CLI 프로필 설정
aws configure --profile ple-platform
# 리전: ap-northeast-2
# IAM Role: AWSPLEPlatformSageMakerRole

# 현재 자격 확인
aws sts get-caller-identity --profile ple-platform

# S3 버킷 접근 확인
aws s3 ls s3://aiops-ple-financial/ --profile ple-platform
```

Docker 환경

```
# 학습 컨테이너 빌드 (1회)
docker build -t ple-training:latest -f containers/training/Dockerfile .

# GPU 접근 테스트
docker run --rm --gpus all ple-training:latest nvidia-smi
```

배포 환경

본 파이프라인은 두 가지 환경에서 동일하게 작동한다:

- **AWS**: SageMaker Training Job (학습) + Lambda/ECS (서빙) + Bedrock (추천사유/에이전트)
- **온프레미스(폐쇄망)**: 로컬 GPU(RTX 4070, 64GB RAM) + Docker + vLLM(Exaone/Qwen)

코드와 config는 동일하며, 환경 변수(`SM_MODEL_DIR` 유무)로 자동 분기된다.

디렉터리 구조 (핵심)

```
aws_ple_for_financial/
├── configs/
│   ├── pipeline.yaml              # 공통 파이프라인 설정 (데이터셋 무관 공유)
│   └── datasets/
│       └── santander.yaml         # 데이터셋별 오버라이드 (id_col, label_cols, 경로 등)
│   └── santander/
│       └── feature_groups.yaml    # 5축 피처 그룹 정의
├── containers/training/
│   ├── train.py                   # SageMaker 학습 진입점
│   └── Dockerfile
├── adapters/
│   └── santander_adapter.py       # raw data → DataFrame
├── core/
│   ├── pipeline/runner.py         # Phase 0 실행
│   ├── training/trainer.py        # PLETrainer
│   ├── config_builder.py          # PLEConfig 단일 진실 소스 (pipeline.yaml + dataset
yaml 병합)
│   └── serving/predict.py         # 추론 서비스
├── scripts/
│   ├── run_local_ablation.py      # 로컬 ablation
│   ├── run_santander_ablation.py  # SageMaker ablation 오케스트레이터
│   ├── run_sagemaker_teacher.py   # SageMaker: 교사 학습 Job 제출
│   ├── run_sagemaker_eval.py      # SageMaker: 평가 Job 제출
│   ├── run_sagemaker_distillation.py  # SageMaker: 증류 Job 제출
│   └── package_source.py          # 소스 패키지 빌드 및 S3 스테이징 (1회 실행 후 재사용)
└── data/benchmark/
    └── benchmark_v2.parquet       # 941K 사용자 데이터
```

Phase 0: 피처 엔지니어링

Phase 0은 원시 데이터를 학습 가능한 텐서로 변환하는 단계이다. CPU 인스턴스에서 실행한다 (GPU 낭비 금지).

실행 흐름

```
Raw Parquet → Adapter → PipelineRunner → Training-ready Artifacts
                                          ├── features.parquet
                                          ├── labels.parquet
                                          ├── sequences/ (3D tensors)
                                          ├── feature_schema.json
                                          ├── feature_stats.json
                                          └── label_stats.json
```

Adapter 실행

Adapter는 raw data를 표준 DataFrame으로 변환한다. 전처리/피처생성/레이블파생은 하지 않는다.

```python
# 로컬 실행 (DuckDB 백엔드)
python -m core.pipeline.runner \
  --config configs/santander/pipeline.yaml \
  --stage adapter \
  --backend duckdb
```

PipelineRunner — 전처리 파이프라인

```python
# 전체 Phase 0 실행
python -m core.pipeline.runner \
  --config configs/santander/pipeline.yaml \
  --feature-groups configs/santander/feature_groups.yaml

# 특정 stage부터 재개 (pipeline_state.json 기반)
python -m core.pipeline.runner \
  --config configs/santander/pipeline.yaml \
  --resume
```

3단계 정규화 (필수 이해)

Stage	처리	대상
1	멱법칙 감지 (skew+kurt → log-log R^2) + `log1p` 복사본 생성	high-skew 컬럼
2	StandardScaler (TRAIN fit only)	continuous 컬럼 (binary 제외)
3	멱법칙 `_log` 복사본은 스케일링 안 함	raw magnitude 보존

주의: Scaler는 반드시 TRAIN split에서만 fit한다. val/test는 transform only.

Generator 입력 라우팅

Generator(GMM, TDA 등)의 입력은 `feature_groups.yaml`의 `input_filter`에서 선언한다.

```
# feature_groups.yaml 예시
generator_params:
  input_filter:
    dtype: continuous        # continuous | all_numeric
    exclude_binary: true     # GMM, TDA용
    min_nunique: 20          # 이산 변수 제외
```

금지: adapter에서 `product_cols`, `synth_cols` 같은 하드코딩 라우팅을 하면 안 된다.

FeatureRouter — Expert별 피처 서브셋 라우팅 (활성화됨)

FeatureRouter는 현재 **활성화** 상태이다. 각 expert는 전체 349D 입력(Phase 0 후 403D) 피처 중 자신에게 지정된 feature group만 입력으로 받는다. `feature_groups.yaml`의 `target_experts` 선언이 실제 런타임 라우팅을 결정한다. expert_routing은 그룹 단위로 동작하며, `build_model()` 시점에 `feature_groups.yaml`에서 자동 빌드된다.

Expert별 입력 차원 (Phase 0 v3/v4):

Expert	입력 차원	주요 소스 그룹
deepfm	168D	State축 전반
temporal_ensemble	139D	Timeseries축 전반
hgcn	27D	merchant_hierarchy (MCC Poincaré L1/L2 27D)
perslay	32D	tda_local 16D + tda_global 16D
causal	161D	Snapshot축 전반
lightgcn	100D	product_hierarchy 32D + graph_collaborative 66D + 기타
optimal_transport	127D	Snapshot/State 혼합

전체 피처는 349D 입력(Phase 0 후 403D)이며, 각 expert는 전체의 부분집합을 입력으로 받는다. FeatureRouter 활성화로 모델 파라미터가 4.77M → 2.8M으로 감소했다.

구현 방식: `target_experts` config에서 읽어 `FeatureRouter`가 `feature_group_ranges`를 참조, expert별로 해당 컬럼 범위를 슬라이싱하여 전달한다. 하드코딩 라우팅은 금지한다.

Phase 0 출력 검증 (Pre-flight Check)

Phase 0 완료 후, 학습 전에 반드시 아래를 확인한다:

```
# 1. feature_stats.json — zero-variance, NaN 비율, 피처 수 확인
python -c "
import json
stats = json.load(open('output/feature_stats.json'))
print(f'피처 수: {stats[\"num_features\"]}')
print(f'Zero-variance: {stats.get(\"zero_variance_cols\", [])}')
print(f'NaN 비율 > 0.5: {[c for c,v in stats[\"nan_ratio\"].items() if v > 0.5]}')
```

```
"

# 2. label_stats.json — class balance, positive rate 확인
python -c "
import json
stats = json.load(open('output/label_stats.json'))
for task, info in stats.items():
    print(f'{task}: {info}')
"

# 3. LeakageValidator (자동 실행됨, 로그 확인)
# train.py 내에서 학습 전 자동 호출 — correlation > 0.95 피처 자동 제거
```

SageMaker Processing Job (클라우드)

```
# Phase 0을 SageMaker CPU 인스턴스에서 실행
python scripts/submit_processing_job.py \
  --config configs/santander/pipeline.yaml \
  --instance-type ml.m5.2xlarge \
  --dry-run  # 먼저 Job 구성 확인
```

Phase 1–3: 학습 + Ablation

학습 아키텍처 개요

```
PLE 2-Phase Training:
  Phase 1 (15 epochs): Joint training — 모든 expert + task tower
  Phase 2 (8 epochs):  Tower fine-tune — shared expert freeze, task head만 학습

핵심 구성요소:
  - 7 shared experts: deepfm, temporal_ensemble, hgcn, perslay,
                      causal, lightgcn, optimal_transport
  - 1 task expert: mlp (태스크별)
  - 13 tasks (4 groups): binary 7 / multiclass 3 / regression 3  (14→13:
deterministic-leakage 태스크 제거)
  - Uncertainty weighting (Kendall et al.)
  - AMP (Mixed Precision) 필수 활성화
```

모드별 실행 방법

모드 1: 로컬 직접 실행 (개발/디버깅)

```
# 소규모 테스트 (50K subsample, 3 epochs)
python containers/training/train.py \
  --config configs/santander/pipeline.yaml \
  --data-dir data/benchmark/ \
  --output-dir output/local_test/ \
  --epochs 3

# 전체 데이터 로컬 테스트 (1 epoch end-to-end 확인 필수)
python containers/training/train.py \
  --config configs/santander/pipeline.yaml \
  --data-dir data/benchmark/ \
  --output-dir output/full_test/ \
  --epochs 1
```

모드 2: Docker 로컬 실행 (SageMaker 환경 재현)

```
# Docker로 SageMaker 환경 동일 재현
docker run --rm --gpus all \
  -v $(pwd)/data:/opt/ml/input/data/training \
  -v $(pwd)/output:/opt/ml/model \
  -v $(pwd)/configs:/opt/ml/input/config \
  ple-training:latest \
  --config /opt/ml/input/config/santander/pipeline.yaml
```

```
# ablation 시나리오 테스트
docker run --rm --gpus all \
  -v $(pwd)/data:/opt/ml/input/data/training \
  -v $(pwd)/output:/opt/ml/model \
  ple-training:latest \
  --removed-feature-groups "tda_global,gmm_clustering" \
  --epochs 3
```

모드 3: SageMaker 클라우드 실행

```
# Step 1: 소스 패키지 빌드 및 S3 스테이징 (1회, 모든 Job에서 재사용)
python scripts/package_source.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml

# Step 2: 교사 학습 Job 제출
python scripts/run_sagemaker_teacher.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --instance-type ml.g4dn.xlarge \
  --use-spot \
  --dry-run  # 먼저 확인

# Step 3: 평가 Job 제출
python scripts/run_sagemaker_eval.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --checkpoint s3://aiops-ple-financial/checkpoints/best/

# Step 4: 증류 Job 제출
python scripts/run_sagemaker_distillation.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --teacher-checkpoint s3://aiops-ple-financial/checkpoints/best/
```

Ablation 실행 (4-Dimension, 23 시나리오)

Ablation 구조

Phase	시나리오 수	내용	인스턴스
0	1	Data Preparation	CPU (ml.m5.2xlarge)
1	16	Feature Group Ablation (bottom-up + top-down)	GPU
2	16	Expert Ablation (bottom-up + top-down)	GPU
3	16	Task × Structure Cross (4 tiers × 4 variants)	GPU
4	2	Best-Config Teacher + Distillation	GPU + CPU
5	1	Analysis + HTML Report	CPU

로컬 Ablation 실행

```
# 로컬 GPU에서 순차 실행 (RTX 4070, ~24시간)
python scripts/run_local_ablation.py \
  --config configs/santander/pipeline.yaml \
  --phase 1  # Feature Group Ablation만

# 전체 Phase 실행
python scripts/run_local_ablation.py \
  --config configs/santander/pipeline.yaml \
  --phase all
```

SageMaker Ablation 실행 (병렬)

```
# SageMaker Spot 4대 병렬 (~4시간)
python scripts/run_santander_ablation.py \
  --config configs/santander/pipeline.yaml \
  --max-parallel 4 \
  --use-spot \
  --dry-run  # 먼저 확인

# 실제 제출 (budget guard 자동 적용: $80 한도)
python scripts/run_santander_ablation.py \
  --config configs/santander/pipeline.yaml \
  --max-parallel 4 \
  --use-spot
```

Ablation Hyperparameters

시나리오별로 다음 HP를 오버라이드할 수 있다:

```
# 피처 그룹 제거
--removed-feature-groups "tda_global,gmm_clustering"

# Expert 제거
--removed-experts "hgcn,perslay"

# 태스크 수 조절
--num-active-tasks 4  # 4/8/11/13

# 구조 변형
--disable-adatt          # adaTT 비활성화
--disable-ple-stacking   # PLE stacking 비활성화 (단일 CGC)
--ple-num-layers 2       # PLE depth: 1/2/3

# Loss weighting 전략
--loss-weighting-strategy uncertainty  # uncertainty/gradnorm/dwa/fixed
```

Ablation 기본 학습 파라미터

`pipeline.yaml`의 `ablation.training_defaults`에서 읽는다:

```yaml
ablation:
  training_defaults:
    epochs: 5
    batch_size: 6144
    learning_rate: 0.0005
    amp: true
    early_stopping_patience: 3
    seed: 42
    num_workers: 2
    pin_memory: true
    drop_last: true
```

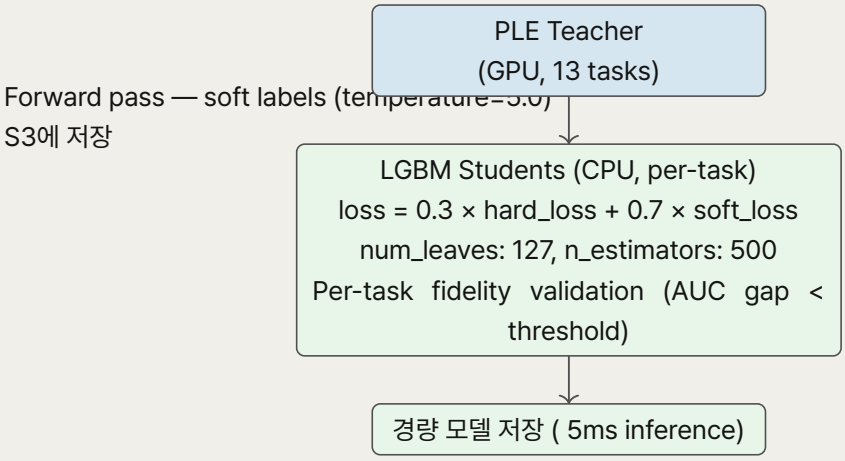

그림 1: Teacher → LGBM Student 지식증류 흐름.

Phase 4: 지식증류

Teacher → LGBM Student

실행 CLI

```
# Step 1: Teacher soft label 생성
python scripts/generate_soft_labels.py \
  --config configs/santander/pipeline.yaml \
  --checkpoint output/best_model/checkpoint.pt \
  --output output/soft_labels/

# Step 2: LGBM Student 학습
python scripts/train_student.py \
  --config configs/santander/pipeline.yaml \
  --soft-labels output/soft_labels/ \
  --output output/student_models/

# Step 3: Fidelity Gate 검증
python scripts/validate_fidelity.py \
  --teacher-metrics output/best_model/eval_metrics.json \
  --student-metrics output/student_models/eval_metrics.json
```

증류 설정 (pipeline.yaml)

```
distillation:
  temperature: 5.0
  alpha: 0.3              # 30% hard + 70% soft
  lgbm:
    num_leaves: 127
```

```yaml
    learning_rate: 0.05
    n_estimators: 500
    min_child_samples: 50
    subsample: 0.8
    colsample_bytree: 0.8
```

Fidelity Gate

증류된 Student가 Teacher 대비 일정 수준 이상의 성능을 유지하는지 검증한다.

- AUC gap < threshold 통과 시 → Student 모델 등록
- 실패 시 → 알림, 수동 검토
- Ablation 시 `--skip-fidelity-gate` 플래그로 비활성화 가능

LGBM 피처 중요도 기반 피처 선택 (증류용)

학습된 LGBM Student 모델의 gain importance로 중요 피처를 선택하여 입력 차원을 축소한다. 교사 모델 IG(Integrated Gradients)는 사용하지 않는다 — 서빙 모델(LGBM) 관점에서의 중요도가 더 적합하고, 교사 IG는 941K 스케일에서 OOM 장애를 유발한다.

```bash
python scripts/run_lgbm_feature_selection.py \
  --config configs/santander/pipeline.yaml \
  --cumulative-gain-threshold 0.95  # 누적 gain 95% 포착
```

Phase 5: 서빙

3단계 확장 아키텍처

단계	구성	비용/월	지연
1단계 (소규모)	API Gateway → Lambda + 메모리 피처	~$0–1	~5ms
2단계 (중규모)	API Gateway → Lambda + DynamoDB	~$100–400	~10ms
3단계 (대규모)	ALB → ECS + Redis + RocksDB + LanceDB	~$360	~5–8ms

Lambda 배포

```
# LGBM Student 모델을 Lambda로 배포
python scripts/deploy_lambda.py \
  --config configs/santander/pipeline.yaml \
  --model-path output/student_models/ \
  --memory-mb 1024 \
  --timeout 30

# 배포 확인
aws lambda invoke \
  --function-name ple-recommend \
  --payload '{"user_id": "test_001", "context": {}}' \
  response.json
```

추론 흐름

```
요청 (user_id, context)
  ↓
① 피처 조회 (메모리 or DynamoDB)      ~0.01ms or ~5ms
  ↓
② 실시간 컨텍스트 결합                ~0.1ms
  ↓
③ LGBM 멀티태스크 추론               ~5ms
  ↓
④ 출력 정규화 + 추천사유 생성         ~0.1ms
  ↓
총: ~5-10ms
```

추천사유 생성 (Reason Generation)

3단계 Interpretability Pipeline:

- **Stage A**: Integrated Gradients + Expert Redundancy CCA + CGC Gate Analysis
- **Stage B**: Multi Interpreter → Template Reason Engine → XAI Quality Evaluator

- **Stage C**: Context Vector Store (RAG) → CPE Scoring → Agentic Orchestrator

A/B 테스트

```yaml
# configs/serving/ab_test.yaml
ab_test:
  enabled: true
  variants:
    - name: control
      model: s3://bucket/models/lgbm-v1/
      weight: 90             # 90% 트래픽
    - name: treatment
      model: s3://bucket/models/lgbm-v2/
      weight: 10             # 10% 트래픽
  evaluation:
    primary_metric: click_through_rate
    min_sample_size: 10000
    significance_level: 0.05
  auto_promote:
    enabled: true
    min_improvement: 0.02    # 2% 이상 개선 시 자동 전환
```

카나리 배포: 5% → 25% → 50% → 100% (이상 감지 시 즉시 롤백)

Config 가이드

모든 파라미터는 YAML config에서 읽는다. Python 코드에 하드코딩 금지.

Split-Config 구조

Config는 두 계층으로 분리한다:

- **`configs/pipeline.yaml`** (공통): 아키텍처, 학습 HP, expert 구성, 태스크 구조, AWS 기본값 — 모든 데이터셋 공유.
- **`configs/datasets/santander.yaml`** (데이터셋별): `id_col`, `date_col`, `label_cols`, `data_path`, 데이터셋 고유 오버라이드.

`config_builder.py`는 두 파일을 병합하여 `PLEConfig`를 구성하는 **단일 진실 소스**이다. 모든 스크립트(`train.py`, `run_sagemaker_*.py`, `run_local_ablation.py`)는 YAML을 직접 읽지 않고 `config_builder.build()`를 호출한다. 데이터셋 고유 변경이 `train.py`나 ablation 스크립트 수정을 요구하지 않음을 보장한다.

```
# config_builder.py 사용 예시
from core.config_builder import build_config
config = build_config(
    pipeline_yaml="configs/pipeline.yaml",
    dataset_yaml="configs/datasets/santander.yaml",
    overrides={}   # CLI 오버라이드 (ablation HP)
)
```

pipeline.yaml 주요 설정

tasks 섹션

```
tasks:
  - name: has_nba          # 태스크 이름 (고유)
    type: binary           # binary | multiclass | regression
    loss: focal            # focal | ce | huber
    loss_params:
      alpha: 0.90          # focal alpha (positive rate 반영)
      gamma: 2.0
    loss_weight: 2.5       # Uncertainty weighting 초기 가중치
    label_col: has_nba     # 레이블 컬럼명
```

training 섹션

```
training:
  batch_size: 2048         # 941K / 2048 = ~460 steps/epoch
  epochs: 50
  learning_rate: 0.0005
  weight_decay: 0.01
  gradient_clip_norm: 5.0
```

```yaml
  seed: 42
  phase1:
    epochs: 15
  phase2:
    epochs: 8
    freeze_shared: true    # shared expert freeze
  early_stopping:
    patience: 7
    metric: val_loss
  scheduler:
    type: cosine
    warmup_epochs: 3
```

data 섹션

```yaml
data:
  id_col: customer_id
  backend: [cudf, duckdb, pandas]  # 우선순위 순서
  temporal_split:
    enabled: true
    date_col: snapshot_date
    gap_days: 1            # 최소 1일 gap (월별 스냅샷)
    train_ratio: 0.7
    val_ratio: 0.15
```

aws 섹션

```yaml
aws:
  region: ap-northeast-2
  s3_bucket: aiops-ple-financial
  instance_type: ml.g4dn.xlarge      # GPU (Phase 1-3)
  cpu_instance_type: ml.m5.2xlarge   # CPU (Phase 0)
  use_spot: true
  max_run_seconds: 43200             # 12시간
```

cold_start 섹션

거래 이력이 부족한 고객(cold start)에 대한 처리를 정의한다. 시퀀스 기반 피처(HMM, Mamba, TDA local)는 이력이 없으면 무의미하므로 0으로 마스킹하고, 인구통계/상품/글로벌 집계 피처는 보존한다.

```yaml
cold_start:
  seq_col: txn_amount_seq            # 이력 깊이 측정 기준 컬럼
  min_txn_count: 3                   # 이 이하 → cold start 플래그
  zero_features_prefix:              # cold start 시 0으로 마스킹할 피처 접두사
    - hmm_states
    - mamba_temporal
    - tda_local
  keep_features_prefix:              # cold start에서도 유지되는 피처 (참고용)
    - prod_
    - synth_
```

```
    - tda_global
    - gmm_
    - graph_
```

동작 원리:

1. `seq_col` 컬럼(LIST 타입)의 길이가 `min_txn_count` 이하이면 해당 고객을 cold start로 분류한다.
2. `zero_features_prefix`에 해당하는 생성 피처를 0으로 대체한다 — 이 피처들은 충분한 시퀀스 없이는 노이즈만 생성한다.
3. `keep_features_prefix`의 피처는 cold start 여부와 무관하게 원본 값을 유지한다.

feature_groups.yaml 핵심 구조

5축 피처 아키텍처:

축	설명	예시 그룹
State	정적 인구통계/계정 속성	demographics, product_holdings
Snapshot	장기 시퀀스 요약	TDA global, HMM, trends
Timeseries	단기 시계열 패턴	TDA local, Mamba, txn
Hierarchy	상품/MCC 트리 임베딩	Poincare embeddings
Item	협업 필터링	LightGCN, product holdings

각 그룹의 핵심 필드:

```yaml
- name: demographics       # 그룹 이름
  group_type: transform    # transform | generator
  columns: [age, income]   # 입력 컬럼
  output_dim: 38           # 출력 차원
  target_experts: [deepfm, mlp]  # 라우팅할 expert
  distill: true            # 증류 대상 여부
```

task_groups 섹션 (GradSurgery / 태스크 유형 프로젝션 실험)

참고: adaTT는 13-task 규모에서 성능 저하. GradSurgery(PCGrad 태스크 유형 프로젝션)를 대안으로 실험하였으나 채택하지 않았다 — PLE 단독 baseline 대비 의미 있는 개선이 없었고 VRAM 오버헤드가 발생했다. 최종 구성은 adaTT와 GradSurgery를 모두 비활성화한다.

```yaml
# grad_surgery 섹션 (adaTT task_groups 대안으로 실험; 프로덕션 미채택)
grad_surgery:
  enabled: true
  task_type_groups:
    binary: [churn_signal, will_acquire_deposits, will_acquire_investments,
             will_acquire_accounts, will_acquire_lending, will_acquire_payments,
             top_mcc_shift]
    multiclass: [nba_primary, segment_prediction, next_mcc]
    regression: [product_stability, cross_sell_count, mcc_diversity_trend]
```

task_relationships (Logit Transfer)

3가지 전이 방식:

방식	설명
`output_concat`	source task의 출력을 target tower 입력에 concat
`hidden_concat`	source task의 hidden representation을 공유
`residual`	source task 출력을 residual connection으로 추가

모니터링

데이터 드리프트

```yaml
# configs/monitoring.yaml
monitoring:
  drift:
    metric: psi               # Population Stability Index
    warning_threshold: 0.1
    critical_threshold: 0.25
    check_frequency: daily
    action_on_critical: trigger_retraining  # 3일 연속 시
```

```bash
# 수동 드리프트 체크
python scripts/check_drift.py \
  --reference data/benchmark/benchmark_v2.parquet \
  --current data/latest/latest.parquet \
  --threshold 0.1
```

모델 성능 모니터링

```yaml
monitoring:
  model:
    metrics: [auc, mae, f1_macro]
    degradation_threshold: 0.05  # 5% 이상 성능 저하
    action: notify               # notify | auto_retrain
```

Fairness 감사

모델 예측의 공정성을 검증한다. segment/gender/age_group별 성능 차이를 모니터링한다.

리니지 추적

3계층 감사 아키텍처:

- **Layer 1** (자동): CloudTrail, S3 Versioning, SageMaker Lineage
- **Layer 2** (반자동): ExperimentTracker, SchemaRegistry, ModelRegistry
- **Layer 3** (명시적): AuditLogger, ComplianceChecker, AccessController

```bash
# 리니지 조회
aws s3 ls s3://aiops-ple-financial/audit/lineage/ --recursive

# SageMaker Experiment 조회
python -c "
```

```
from sagemaker.analytics import ExperimentAnalytics
analytics = ExperimentAnalytics(experiment_name='santander_ple')
df = analytics.dataframe()
print(df[['TrialName', 'auc_roc', 'val_loss']].head(20))
"
```

Champion/Challenger 평가 (오프라인 게이트)

`scripts/submit_pipeline.py::_decide_promotion`이 아래 순서로 자동 판정하며, 모든 결과는 `AuditLogger.log_model_promotion`(HMAC + hash chain)으로 S3 WORM 감사 로그에 기록됩니다.

```
① --force-promote 플래그 존재?       → promote (operator override)
② 현 champion 없음?                  → promote (bootstrap)
③ fidelity_summary.failed > 0?       → reject (safety floor)
④ ModelCompetition.evaluate():
   - primary metric 개선 ≥ 0.5%
   - secondary metric 하락 ≤ 2%
   - (선택) paired bootstrap 유의성
   승인                               → promote
   거부                               → register only (promoted=False)
```

| 조건 | 결과 | audit decision | |—|—|—| | `--force-promote` | 항상 승격 (수동 override) | `force_promote` | | champion 없음 | bootstrap 승격 | `bootstrap` | | fidelity 실패 ≥ 1건 | 등록만 (안전 floor) | `reject` | | Competition 승인 | 자동 승격 | `promote` | | Competition 거부 | 등록만 (승격 안 함) | `reject` |

온라인 게이트(`ModelMonitor.evaluate_champion_challenger`, DynamoDB prediction log 기반 two-proportion z-test)는 실 트래픽 누적 후 별도 스케줄 트리거로 활성화 예정.

문제 해결 (FAQ)

Phase 2 학습 중 NaN Loss 발생

원인: FocalLoss에 sigmoid 적용된 값이 다시 들어가는 double-sigmoid 문제, 또는 label에 NaN이 포함.

해결:

```
# 1. FocalLoss에는 pre-activation logits를 전달 (sigmoid 중복 방지)
# 2. 어떤 태스크에서 NaN이 발생했는지 로그 확인
grep "NaN.*loss" output/training.log

# 3. label NaN 비율 확인
python -c "
import pyarrow.parquet as pq
labels = pq.read_table('output/labels.parquet')
for col in labels.column_names:
    null_count = labels.column(col).null_count
    print(f'{col}: {null_count}/{len(labels)} ({null_count/len(labels)*100:.1f}%)')
"
```

GPU Utilization 낮음 (< 50%)

원인: DataLoader bottleneck, 작은 batch_size, 또는 CPU 전처리 병목.

해결:

```
# 1. batch_size 증가 (VRAM 허용 범위 내)
# 941K 데이터 → batch_size: 5632 권장

# 2. DataLoader num_workers 확인
# pipeline.yaml ablation.training_defaults.num_workers: 2

# 3. pin_memory 활성화 확인
# pipeline.yaml ablation.training_defaults.pin_memory: true

# 4. GPU 메모리 사용량 확인 (매 epoch 로깅됨)
grep "gpu_memory" output/training.log
```

Label Leakage 의심

원인: 시퀀스의 마지막 timestep이 레이블과 겹치거나, scaler가 val/test 데이터로 fit됨.

해결:

```
# 1. LeakageValidator 로그 확인
grep "LeakageValidator" output/training.log
grep "correlation.*>.*0.95" output/training.log

# 2. 시퀀스 truncate 설정 확인
# pipeline.yaml sequences.product_sequences.truncate_last: 1
# → month 17 (label month) 제거, month 1-16만 사용

# 3. prod_* 컬럼이 seq_* month 16에서 재계산되는지 확인
# pipeline.yaml data.preprocessing.leakage_prevention.recompute_prod_from_seq: true
```

SageMaker Job Timeout

원인: max_run_seconds 부족 또는 데이터 크기 과대 추정.

해결:

```
# 1. max_run_seconds 확인
# pipeline.yaml aws.max_run_seconds: 43200 (12시간)

# 2. 소규모 테스트로 예상 시간 파악 후 설정
# 50K × 3 epochs → 실측 시간 × (941K/50K) × (full_epochs/3) 추정

# 3. Spot 중단과 알고리즘 에러 구분
# SageMaker SecondaryStatusTransitions 확인
aws sagemaker describe-training-job \
  --training-job-name <job-name> \
  --query 'SecondaryStatusTransitions'
```

Spot Instance 중단 빈번

원인: 동시 4대 초과 시 같은 AZ 경쟁으로 중단 빈도 급증.

해결:

```
# 1. 동시 Spot 인스턴스 4대 이하로 제한
# scripts/run_santander_ablation.py --max-parallel 4

# 2. max_wait 설정: max_run + 1시간
# 10시간 대기는 낭비 — max_wait = max_run_seconds + 3600

# 3. 체크포인트 확인 (매 epoch S3 저장)
aws s3 ls s3://aiops-ple-financial/checkpoints/<job-name>/
```

Gradient Norm 폭발

원인: learning rate 과다 또는 데이터 이상치.

해결:

```
# gradient_clip_norm 확인 (pipeline.yaml training.gradient_clip_norm: 5.0)
# clip 임계값의 10배 초과 시 경고 로그 확인
grep "gradient.*norm.*exceed" output/training.log

# learning_rate 하향 조정: 0.0005 → 0.0001
```

비용 관리

Spot Instance 전략

항목	On-Demand	Spot
g4dn.xlarge 시간당	$0.526	~$0.16 (70% 절감)
50 epochs (~4시간)	$2.10	~$0.64
23 시나리오 ablation (14 joint + 9 structure cross)	~$100	~$30

비용 확인 CLI

```
# 현재 월 비용 확인 (SageMaker 제출 전 필수)
aws ce get-cost-and-usage \
  --time-period Start=$(date -d "$(date +%Y-%m-01)" +%Y-%m-%d),End=$(date +%Y-%m-%d)
\
  --granularity MONTHLY \
  --metrics BlendedCost \
  --filter '{"Dimensions":{"Key":"SERVICE","Values":["Amazon SageMaker"]}}'
```

Budget Guard

`pipeline.yaml`의 `ablation.budget_limit: 80.0` (USD)을 초과하면 ablation이 자동 중단된다.

```
# 예산 설정 확인
grep "budget_limit" configs/santander/pipeline.yaml
# ablation.budget_limit: 80.0
```

비용 최적화 체크리스트

1. **ProfilerReport 비활성화**: `disable_profiler=True` (SageMaker estimator)
2. **AMP 활성화**: g4dn T4 GPU에서 ~2배 속도 향상
3. **batch_size 최적화**: 941K → 5632 권장
4. **Spot 동시 4대 이하**: AZ 경쟁 방지
5. **max_wait = max_run + 1시간**: 과도한 대기 방지
6. **source 패키지 1회 빌드**: 모든 Job에서 재사용
7. **Phase 0은 CPU 인스턴스**: GPU 낭비 방지
8. **Warm Pool 활성화**: 연속 Job 간 인스턴스 재사용
9. **S3 결과 존재 확인**: 중복 Job 방지
10. **실험 후 실제 비용 확인**: 추정치 대비 2배 이상 차이 시 원인 분석

인프라 비용 비교

방식	월 비용 (추정)	비고
K8s 자체 구축 (GPU 4장)	$8,000–15,000	하드웨어 감가 + 인력
SageMaker On-Demand	$500–2,000	주 1회 학습 + 서빙
SageMaker Spot + Lambda	$200–800	최적화 시

오케스트레이션 비용

```
Airflow (기존): 4 컨테이너 상시 → ~$200-300/월
Step Functions (AWS): 5 상태 머신 × 주 2회 → ~$0.002/월 (사실상 무료)
```

부록: 운영 규칙 요약

절대 금지사항

- SageMaker에서 코드 디버깅 금지 (제출당 $0.50+ 비용)
- `train.py`에 전처리 코드 금지
- Adapter에 generator 호출 하드코딩 금지
- `FEATURE_GROUP_COLUMN_PREFIXES` 같은 하드코딩 매핑 금지
- pandas 직접 사용 지양 (cuDF → DuckDB → pandas fallback 순서)
- 실험 결과 검증 없이 다음 Phase 진행 금지

관심사 분리 원칙

모듈	역할	하면 안 되는 것
Adapter	raw data → standardized DataFrame	전처리, 피처생성, 레이블파생
PipelineRunner	전처리 → 피처생성 → 정규화 → 텐서 저장	모델 학습
train.py	데이터 로드 → 모델 빌드 → 학습	전처리 (fillna, scaler 등)
Ablation script	시나리오 오케스트레이션	시나리오/expert 목록 하드코딩

개발 순서 (필수 준수)

1. 로컬 50K subsample 테스트 (end-to-end 성공 확인)
2. 로컬 전체 데이터 1 epoch 테스트
3. Docker 환경 재현 테스트
4. SageMaker `--dry-run` 확인
5. SageMaker 제출

체크포인트 재개 (Checkpoint Resume)

학습 체크포인트는 매 epoch마다 S3에 저장된다. trainer는 최신 체크포인트를 찾을 때 `.pt`와 `.pth` 두 패턴을 모두 검사한다 (파일 패턴 불일치 수정 완료).

Epoch 카운팅: 재개 후 epoch 번호는 저장된 epoch 인덱스에서 이어진다 (0으로 초기화 안 함). `trainer.py`는 `checkpoint["epoch"]`를 읽어 `start_epoch = checkpoint["epoch"] + 1`로 설정한다.

```
# S3의 최신 체크포인트에서 학습 재개
python scripts/run_sagemaker_teacher.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --resume  # output S3 경로에서 최신 .pt/.pth 자동 탐지
```

Pipeline State 자동 복구

`pipeline_state.json`에서 완료된 stage를 추적한다. 중단 시 `--resume` 플래그로 이어서 실행한다.

```
{
  "completed_stages": ["adapter", "temporal_prep", "schema",
    "encryption", "features", "labels", "leakage",
    "sequences", "dataloader", "training"],
  "artifacts": {
    "features": {"path": "features.parquet", "rows": 941132, "dim": 512}
  }
}
```

운영/감사 에이전트 연계

파이프라인 각 스테이지 완료 시 `_PipelineState.mark_complete()` 콜백을 통해 OpsAgent에 이벤트가 전달된다. OpsAgent는 7개 체크포인트(CP1 인제스천 ~ CP7 A/B테스트)를 점검하고, AuditAgent는 추천사유 품질(3-Tier 검증)과 규제 적합성을 감사한다.

변경 사항(코드, 설정, 모델, 데이터)은 Push/Pull 이중 채널로 감지되어 영향 받는 파트의 체크리스트가 자동 재실행된다.

상세 설계: Design Document 11 (`docs/design/11_ops_audit_agent.typ`)